CODIGO 1 — CLASE ABSTRACTA

```java
// Clase Madre Abstracta (Trabajo 1)
public abstract class ItemRecompensa {
    private String nombre;
    private int costoPuntos;

    public ItemRecompensa(String nombre, int costoPuntos) {
        this.nombre = nombre;
        this.costoPuntos = costoPuntos;
    }

    // Método abstracto que las subclases deben implementar obligatoriamente
    public abstract void aplicarEfecto();

    // Métodos Getters
    public String getNombre() { return nombre; }
    public int getCostoPuntos() { return costoPuntos; }
}
```

CODIGO 2 — SUB CLASE 1

```java
// Subclase 1: ItemAvatar (Trabajo 1)
public class ItemAvatar extends ItemRecompensa {
    private String tipoSkin;

    public ItemAvatar(String nombre, int costoPuntos, String tipoSkin) {
        super(nombre, costoPuntos);
        this.tipoSkin = tipoSkin;
    }

    @Override
    public void aplicarEfecto() {
        System.out.println("Desbloqueando skin de avatar tipo: " + tipoSkin);
    }
}
```

CODIGO 3 — SUB CLASE 2

```java
// Subclase 2: ItemPaseAcademico (Trabajo 1)
public class ItemPaseAcademico extends ItemRecompensa {
    private int duracionDias;

    public ItemPaseAcademico(String nombre, int costoPuntos, int duracionDias) {
        super(nombre, costoPuntos);
        this.duracionDias = duracionDias;
    }

    @Override
```

```
    public void aplicarEfecto() {
        System.out.println("Otorgando pase académico especial por " + duracionDias + "
días.");
    }
}
```

EXPLICACIÓN CÓDIGO 1

// Clase Madre Abstracta (Trabajo 1)

Comentario que indica que esta es la clase "madre" (base) abstracta de la que heredarán todos los tipos de recompensa.

public abstract class ItemRecompensa {

Declara la clase ItemRecompensa como abstract: esto impide crear objetos directamente de ella (new ItemRecompensa(...) daría error); solo sirve como plantilla para sus subclases.

private String nombre;

Declara el atributo privado que guardará el nombre del ítem.

private int costoPuntos;

Declara el atributo privado que guardará el costo en puntos del ítem.

public ItemRecompensa(String nombre, int costoPuntos) {

Constructor de la clase abstracta: aunque no se pueda instanciar ItemRecompensa directamente, este constructor SÍ se ejecuta cuando una subclase lo invoca con super(...).

this.nombre = nombre;

Asigna el parámetro nombre al atributo nombre de este objeto.

this.costoPuntos = costoPuntos;

Asigna el parámetro costoPuntos al atributo costoPuntos de este objeto.

}

Cierra el constructor.

// Método abstracto...

Comentario que explica el propósito del método abstracto que sigue.

public abstract void aplicarEfecto();

Declara un método abstracto (sin cuerpo, termina en punto y coma): obliga a que CUALQUIER subclase de ItemRecompensa implemente su propia versión de aplicarEfecto().

// Métodos Getters

Comentario que anuncia los métodos getters.

public String getNombre() { return nombre; }

Método getter que devuelve el nombre del ítem (permite leerlo desde fuera de la clase, ya que el atributo es private).

public int getCostoPuntos() { return costoPuntos; }

Método getter que devuelve el costo en puntos del ítem.

}

Cierra la clase ItemRecompensa.

EXPLICACIÓN CÓDIGO 2

// Subclase 1: ItemAvatar (Trabajo 1)

Comentario que anuncia el inicio de la primera subclase: ItemAvatar.

public class ItemAvatar extends ItemRecompensa {

Declara la clase ItemAvatar, que hereda (extends) de ItemRecompensa: por eso tiene nombre, costoPuntos y debe implementar aplicarEfecto().

private String tipoSkin;

Declara un atributo propio de ItemAvatar (no existe en la clase base): el tipo de skin.

public ItemAvatar(String nombre, int costoPuntos, String tipoSkin) {

Constructor de ItemAvatar, que recibe además el tipo de skin.

super(nombre, costoPuntos);

Llama al constructor de la clase padre (ItemRecompensa) para inicializar nombre y costoPuntos.

this.tipoSkin = tipoSkin;

Asigna el parámetro tipoSkin al atributo propio de esta subclase.

}

Cierra el constructor de ItemAvatar.

@Override

Anotación @Override: le indica a Java (y a quien lea el código) que este método sobrescribe la versión abstracta definida en ItemRecompensa.

public void aplicarEfecto() {

Implementación concreta de aplicarEfecto() específica para ItemAvatar (obligatoria porque la clase base la declaró como abstracta).

System.out.println("Desbloqueando skin...");

Muestra en consola un mensaje propio de este tipo de ítem, usando el atributo tipoSkin.

}

Cierra el método aplicarEfecto() de ItemAvatar.

}

Cierra la clase ItemAvatar.

EXPLICACIÓN CÓDIGO 3

Subclase 2: ItemPaseAcademico (Trabajo 1)

Comentario que anuncia el inicio de la segunda subclase: ItemPaseAcademico.

public class ItemPaseAcademico extends ItemRecompensa {

Declara la clase ItemPaseAcademico, que también hereda de ItemRecompensa.

private int duracionDias;

Declara un atributo propio de esta subclase: la duración en días del pase.

public ItemPaseAcademico(String nombre, int costoPuntos, int duracionDias) {

Constructor de ItemPaseAcademico, que recibe además la duración en días.

super(nombre, costoPuntos);

Llama al constructor de la clase padre para inicializar nombre y costoPuntos.

this.duracionDias = duracionDias;

Asigna el parámetro duracionDias al atributo propio de esta subclase.

}

Cierra el constructor de ItemPaseAcademico.

@Override

Anotación @Override: indica que este método sobrescribe la versión abstracta de la clase base.

public void aplicarEfecto() {

Implementación concreta de aplicarEfecto() específica para ItemPaseAcademico: distinta a la de ItemAvatar, aunque el método se llama igual.

System.out.println("Otorgando pase académico...");

Muestra en consola un mensaje propio de este tipo de ítem, usando el atributo duracionDias.

}

Cierra el método aplicarEfecto() de ItemPaseAcademico.

}

Cierra la clase ItemPaseAcademico.

CÓDIGO 4 — POLIMORFISMO CON ARREGLO

Se crea un arreglo del tipo abstracto ItemRecompensa y se llenan sus posiciones con objetos de distintas subclases. Al recorrer el arreglo y llamar a aplicarEfecto(), Java ejecuta la versión correcta de cada objeto (polimorfismo dinámico).

Código: PrincipalPrueba.java

```
public class PrincipalPrueba {
    public static void main(String[] args) {
        // Creación de un arreglo polimórfico de tipo ItemRecompensa (Trabajo 2)
        ItemRecompensa[] inventarioTienda = new ItemRecompensa[2];

        inventarioTienda[0] = new ItemAvatar("Skin Guerrero Cyber", 150, "Legendario");
        inventarioTienda[1] = new ItemPaseAcademico("Pase de Prórroga", 300, 7);

        System.out.println("--- EJECUTANDO EFECTOS CON POLIMORFISMO ---");

        // Recorriendo el arreglo e invocando el método de forma polimórfica (Trabajo 2)
        for (ItemRecompensa item : inventarioTienda) {
            System.out.println("Ítem: " + item.getNombre() + " | Costo: " +
item.getCostoPuntos() + " pts");
            item.aplicarEfecto(); // Comportamiento dinámico según la subclase
            System.out.println("-------------------------------------------------");
        }
    }
}
```

```
--- EJECUTANDO EFECTOS CON POLIMORFISMO ---
Ítem: Skin Guerrero Cyber | Costo: 150 pts
Desbloqueando skin de avatar tipo: Legendario
-------------------------------------------------
Ítem: Pase de Prórroga | Costo: 300 pts
Otorgando pase académico especial por 7 días.
-------------------------------------------------
PS E:\Semestre IV\Taller de lenguajes de programacion\Codigos>
```

EXPLICACIÓN LÍNEA POR LÍNEA — CÓDIGO 4

public class PrincipalPrueba {

Declara la clase PrincipalPrueba, donde se probará el polimorfismo (esta clase no hereda de ItemRecompensa, solo la usa).

public static void main(String[] args) {

Punto de entrada del programa: aquí comienza la ejecución cuando se corre esta clase.

// Creación de un arreglo polimórfico...

Comentario que explica qué es un arreglo polimórfico y para qué se usa a continuación.

ItemRecompensa[] inventarioTienda = new ItemRecompensa[2];

Declara un arreglo de tipo ItemRecompensa (la clase abstracta) con espacio para 2 elementos: aunque el tipo declarado es el padre, cada casilla podrá guardar cualquier subclase.

inventarioTienda[0] = new ItemAvatar(...);

Guarda un objeto ItemAvatar en la primera posición del arreglo, aunque el arreglo es de tipo ItemRecompensa: esto es posible porque ItemAvatar ES-UN ItemRecompensa (herencia).

inventarioTienda[1] = new ItemPaseAcademico(...);

Guarda un objeto ItemPaseAcademico en la segunda posición del arreglo: un tipo completamente distinto, pero también válido porque también extiende ItemRecompensa.

System.out.println("--- EJECUTANDO EFECTOS...");

Muestra en consola un encabezado antes de comenzar el recorrido.

// Recorriendo el arreglo...

Comentario que explica que el siguiente bucle es donde ocurre el polimorfismo.

for (ItemRecompensa item : inventarioTienda) {

Recorre el arreglo con un for-each: en cada vuelta, item es tratado como un ItemRecompensa genérico, sin importar si en realidad es un ItemAvatar o un ItemPaseAcademico.

System.out.println("Ítem: " + item.getNombre()...);

Muestra en consola el nombre y el costo del ítem actual, usando los métodos getters heredados de ItemRecompensa.

item.aplicarEfecto();

Llama a aplicarEfecto() sobre item: aquí ocurre el polimorfismo real — Java ejecuta automáticamente la versión de ItemAvatar o de ItemPaseAcademico según el tipo real del objeto, no según el tipo declarado del arreglo.

System.out.println("----------");

Muestra una línea separadora en consola entre un ítem y el siguiente.

}

Cierra el bucle for-each.

}

Cierra el método main.

}

Cierra la clase PrincipalPrueba.

Diferencia entre las dos partes del trabajo

Código 1 al 3: Define la jerarquía de clases (clase abstracta + subclases con @Override). Aquí se ve la herencia y la obligación de implementar el método abstracto.

Código 4: Usa un arreglo del tipo padre para guardar objetos de distintas subclases y demuestra el polimorfismo al llamar al mismo método (aplicarEfecto) con comportamientos distintos.